

```

GNU nano 7.2
#include <stdio.h>
#include <string.h>

int main() {
    char command[100];

    while (1) {
        printf("myshell> ");
        fgets(command, sizeof(command), stdin);

        command[strcspn(command, "\n")] = '\0';

        if (strcmp(command, "exit") == 0) {
            printf("Exiting shell...\n");
            break;
        }

        if (strlen(command) > 0) {
            printf("You entered: %s\n", command);
        }
    }

    return 0;
}

```

https://abuse4u.com/engage/secure-kubernetes-at-the-edge

This message is shown once a day. To disable it please create the /home/teja/.hushlogin file.

```

teja@LAPTOP-FGDPB110:~$ nano copy.c
teja@LAPTOP-FGDPB110:~$ |

```

```

OSLab OSSP Week3 copy.c process process.c process.c.save process.c.save.1
teja@LAPTOP-FGDPB110:~$ cat File1.txt
cat: File1.txt: No such file or directory
teja@LAPTOP-FGDPB110:~$ cat File2.txt
cat: File2.txt: No such file or directory
teja@LAPTOP-FGDPB110:~$ Hello, this is File 1.
This content will be copied to File 2.
Hello,: command not found
This: command not found
teja@LAPTOP-FGDPB110:~$ gcc copy.c -o copy
teja@LAPTOP-FGDPB110:~$ ./copy
myshell> cat File2.txt
You entered: cat File2.txt
myshell>

```

Description:

In this task, we use the system calls `open()`, `read()`, `write()`, and `close()`. First, we open `File1.txt` for reading. Then, we read the content from the file and write it into `File2.txt`. Finally, we close both files.

The main system calls used are open(), read(), write(), and close().

Result:

The contents of File1.txt were successfully copied to File2.txt.

TASK 3

```
GNU nano 7.2 task3.c *
{
    if (length < 99)
    {
        input[length] = ch;
        length++;
        putchar(ch);
    }
}

printf("\nYou entered: %s\n", input);

return 0;
}
```

```
myshell> ^C
teja@LAPTOP-FGDPB110:~$ nano task3.c
teja@LAPTOP-FGDPB110:~$ gcc task3.c -o task3
teja@LAPTOP-FGDPB110:~$ ./task3
Enter command: hello
hello
You entered: hello
teja@LAPTOP-FGDPB110:~$ |
```

o capture keyboard input and handle the Backspace and Enter keys.

Theory:

In this task, the program takes input from the keyboard and stores it in an input buffer. Each character entered by the user is stored in the buffer.

The **Backspace key** is used to remove the last character entered. The **Enter key** is used to finish the input. After pressing Enter, the program processes the complete command and displays the entered text.

This task helps us understand how keyboard input is captured, how the input buffer is managed, and how multi-character commands are handled.

Result:

The keyboard input was successfully captured, and the Backspace and Enter keys were handled

TASK 4

GNU nano 7.2

```
{
    if (length < 99)
    {
        input[length] = ch;
        length++;
        putchar(ch);
    }
}

printf("\nYou entered: %s\n", input);

return 0;
```

mysheet> C

```
teja@LAPTOP-FGDPB110:~$ nano task3.c
teja@LAPTOP-FGDPB110:~$ gcc task3.c -o task3
teja@LAPTOP-FGDPB110:~$ ./task3
Enter command: hello
hello
You entered: hello
teja@LAPTOP-FGDPB110:~$ nano history.c
teja@LAPTOP-FGDPB110:~$ gcc history.c -o history
teja@LAPTOP-FGDPB110:~$ ./history
Enter command: ossp
ossp
You entered: ossp
teja@LAPTOP-FGDPB110:~$ |
```

```
1  c++tcvgvgvvvvvvvvv
2  clear
3  sudo apt update
4  sudo apt install build-essential installing:
5  gcc --version
6  gcc process . c -o process
7  ./process
8  clear
9  mkdir OSSP
10 cd OSSP
11 git init
12 mkdir src include obj bin docs tests scripts assets
13 touch .gitignore README.md LICENSE Makefile
14 touch src/main.c
15 touch src/shell.c
16 touch src/parser.c
17 touch src/executor.c
18 touch src/builtin.c
19 touch include/shell.h
20 touch include/parser.h
21 touch include/executor.h
22 touch include/builtin.h
23 touch docs/design.md
24 touch docs/report.pdf
25 touch tests/test_parser.c
26 touch tests/test_executor.c
27 touch scripts/run.sh
28 touch assets/architecture.png
29 tree
30 sudo apt install tree
31 tree
32 sudo apt install tree
33 sudo apt install build-essential
34 gcc --version
35 nano process.c
36 sudo apt install tree
37 sudo apt install build-essential
38 nano process.c
39 pwd
40 clear
```

```
132  lscpu
133  history
134  clear
135  echo hello
136  bc
137  is
138  ls
139  ls -l
140  cd Documents
141  cd ..
142  mkdir OSLab
143  rmdir Test
144  cd ~
145  pwd
146  mkdir OSLab
147  mkdir Test
148  rmdir Test
149  ls
150  pwd
151  mkdir OSLab
152  mkdir Test
153  rmdir Test
154  ls
155  nano copy.c
156  ./copy
157  ls
158  cat File1.txt
159  cat File2.txt
160  Hello, this is File 1.
161  This content will be copied to File 2.
162  gcc copy.c -o copy
163  ./copy
164  nano task3.c
165  gcc task3.c -o task3
166  ./task3
167  nano history.c
168  gcc history.c -o history
169  ./history
170  gcc history.c -o history -lreadline
171  history
```

TASK 5

```

GNU nano 7.2
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

struct Node {
    char *data;
    struct Node *next;
};

int main()
{
    int size = 10;
    char *buffer;
    struct Node *head = NULL;
    struct Node *newNode;

    /* Dynamic buffer allocation */
    buffer = (char *)malloc(size * sizeof(char));

    if (buffer == NULL)
    {
        printf("Memory allocation failed\n");
        return 1;
    }

    printf("Enter a string: ");
    fgets(buffer, size, stdin);

    /* Resize buffer */
    buffer = (char *)realloc(buffer, 100 * sizeof(char));

    if (buffer == NULL)
    {
        printf("Memory reallocation failed\n");
        return 1;
    }
}

```

```

Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
The following package was automatically installed and is no longer required:
  libglapi-mesa
Use 'sudo apt autoremove' to remove it.
The following additional packages will be installed:
  gdb libbabeltrace1 libc6-dbg libdebuginfod-common libdebuginfod1t64 libipt2 libsource-highlight-common libsource-highlight4t64
Suggested packages:
  gdb-doc gdbserver valgrind-dbg valgrind-mpi kcache grind alleyoop valkyrie
The following NEW packages will be installed:
  gdb libbabeltrace1 libc6-dbg libdebuginfod-common libdebuginfod1t64 libipt2 libsource-highlight-common libsource-highlight4t64 valgrind
0 upgraded, 9 newly installed, 0 to remove and 15 not upgraded.
Need to get 27.0 MB of archives.
After this operation, 103 MB of additional disk space will be used.
Do you want to continue? [Y/n] y
Get:1 http://archive.ubuntu.com/ubuntu noble-updates/main amd64 libdebuginfod-common all 0.190-1.1ubuntu0.1 [14.6 kB]
Get:2 http://archive.ubuntu.com/ubuntu noble/main amd64 libbabeltrace1 amd64 1.5.11-3build3 [164 kB]
Get:3 http://archive.ubuntu.com/ubuntu noble-updates/main amd64 libdebuginfod1t64 amd64 0.190-1.1ubuntu0.1 [17.1 kB]

```

```

(Reading database ... 45111 files and directories currently installed.)
Preparing to unpack .../0-libdebuginfod-common_0.190-1.1ubuntu0.1_all.deb ...
Unpacking libdebuginfod-common (0.190-1.1ubuntu0.1) ...
Selecting previously unselected package libbabeltrace1:amd64.
Preparing to unpack .../1-libbabeltrace1_1.5.11-3build3_amd64.deb ...
Unpacking libbabeltrace1:amd64 (1.5.11-3build3) ...
Selecting previously unselected package libdebuginfod1t64:amd64.
Preparing to unpack .../2-libdebuginfod1t64_0.190-1.1ubuntu0.1_amd64.deb ...
Unpacking libdebuginfod1t64:amd64 (0.190-1.1ubuntu0.1) ...
Selecting previously unselected package libipt2.
Preparing to unpack .../3-libipt2_2.0.6-1build1_amd64.deb ...
Unpacking libipt2 (2.0.6-1build1) ...
Selecting previously unselected package libsource-highlight-common.
Preparing to unpack .../4-libsource-highlight-common_3.1.9-4.3build1_all.deb ...
Unpacking libsource-highlight-common (3.1.9-4.3build1) ...
Selecting previously unselected package libsource-highlight4t64:amd64.
Preparing to unpack .../5-libsource-highlight4t64_3.1.9-4.3build1_amd64.deb ...
Unpacking libsource-highlight4t64:amd64 (3.1.9-4.3build1) ...
Selecting previously unselected package gdb.
Preparing to unpack .../6-gdb_15.1-1ubuntu1~24.04.1_amd64.deb ...
Unpacking gdb (15.1-1ubuntu1~24.04.1) ...
Selecting previously unselected package libc6-dbg:amd64.
Preparing to unpack .../7-libc6-dbg_2.39-0ubuntu8.8_amd64.deb ...
Unpacking libc6-dbg:amd64 (2.39-0ubuntu8.8) ...
Selecting previously unselected package valgrind.
Preparing to unpack .../8-valgrind_1%3a3.22.0-0ubuntu3_amd64.deb ...
Unpacking valgrind (1:3.22.0-0ubuntu3) ...
Setting up libdebuginfod-common (0.190-1.1ubuntu0.1) ...
Setting up libsource-highlight-common (3.1.9-4.3build1) ...
Setting up libc6-dbg:amd64 (2.39-0ubuntu8.8) ...
Setting up libipt2 (2.0.6-1build1) ...
Setting up libbabeltrace1:amd64 (1.5.11-3build3) ...
Setting up valgrind (1:3.22.0-0ubuntu3) ...
Setting up libdebuginfod1t64:amd64 (0.190-1.1ubuntu0.1) ...
Setting up libsource-highlight4t64:amd64 (3.1.9-4.3build1) ...
Setting up gdb (15.1-1ubuntu1~24.04.1) ...
Processing triggers for man-db (2.12.0-4build2) ...
Processing triggers for libc-bin (2.39-0ubuntu8.8) ...

```

```

teja@LAPTOP-FGDPB110:~$ valgrind --leak-check=full ./memory
==1018== Memcheck, a memory error detector
==1018== Copyright (C) 2002-2022, and GNU GPL'd, by Julian Seward et al.
==1018== Using Valgrind-3.22.0 and LibVEX; rerun with -h for copyright info
==1018== Command: ./memory
==1018==
Enter a string: hello
Entered text: hello

Linked list data: hello

Memory released successfully.
==1018==
==1018== HEAP SUMMARY:
==1018==    in use at exit: 0 bytes in 0 blocks
==1018==   total heap usage: 6 allocs, 6 frees, 2,181 bytes allocated
==1018==
==1018== All heap blocks were freed -- no leaks are possible
==1018==
==1018== For lists of detected and suppressed errors, rerun with: -s
==1018== ERROR SUMMARY: 0 errors from 0 contexts (suppressed: 0 from 0)
teja@LAPTOP-FGDPB110:~$ |

```

To allocate memory dynamically, resize buffers, manage linked lists, release memory correctly, and verify memory usage using Valgrind.

Theory:

In this task, the program uses dynamic memory allocation to create buffers when memory is needed. The allocated memory can be resized according to the requirement. The program also manages data using a linked list.

Proper memory management is important to avoid buffer overflow and memory leaks. After completing the required operations, the allocated memory is released using the appropriate functions.

Finally, the program is checked using the **Valgrind** tool to verify that memory is being used correctly and that there are no memory leaks.

Result:

Dynamic memory was allocated and managed successfully, the linked-list data was handled correctly, and the allocated memory was released. The program can be verified using Valgrin